Processamento de Transações

Adaptado dos
materiais da professora Raqueline Penteado

Introdução

- **Transação**
  - Unidade lógica de processamento de banco de dados.

- **Importância**
  - Uso em diferentes contextos, tais como reservas aéreas, compras online, sistemas bancários...

- **Tratamentos Necessários - Sistema Gerenciador de Banco de Dados (SGBD)**
  - Multiusuários
    - Concorrência intercalada
      - Plano de execução
        - Restaurabilidade
        - Serialidade

Transação

- **Propriedades desejáveis de uma transação**
  - **Atomicidade**
    - Unidade atômica de processamento.
  - **Consistência**
    - O banco de dados deve continuar consistente após a execução de uma transação.
  - **Isolamento**
    - Uma transação deve ser executada como se estivesse isolada das demais transações.
  - **Durabilidade**
    - Todas as alterações provocadas por uma transação devem ser persistidas no banco de dados.

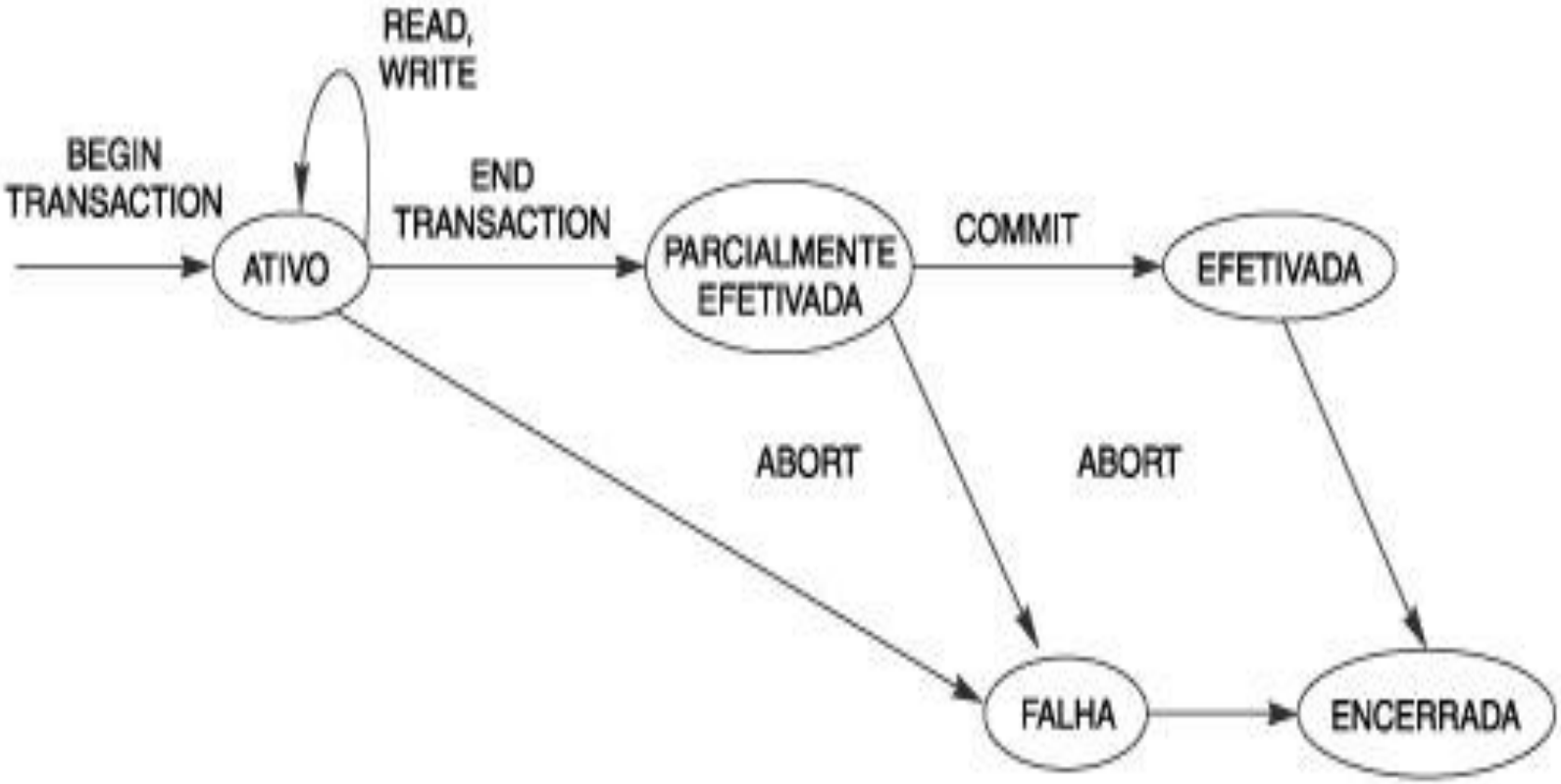
Introdução

- **Exemplo de Transação**

```
BEGIN TRANSACTION;
UPDATE CONTA 123 {SALDO := SALDO - 100};
IF ocorreu algum erro THEN
   GO TO UNDO;
END IF;
UPDATE CONTA 456 {SALDO := SALDO + 100};
IF ocorreu algum erro THEN
   GO TO UNDO;
END IF;
COMMIT;
GO TO FINISH;
UNDO:
   ROLLBACK;
FINISH:
   RETURN;
END TRANSACTION;
```

Transação

- **Explicação com Diagrama de Transição de Estado**



Concorrência

- **Descontrole da execução das operações**
  - Problemas de inconsistência no BD.

- **Problemas**
  - Atualização perdida
  - Atualização temporária
  - Sumário incorreto

Concorrência

- **Exemplo de transações que podem ser concorrentes**

(a)

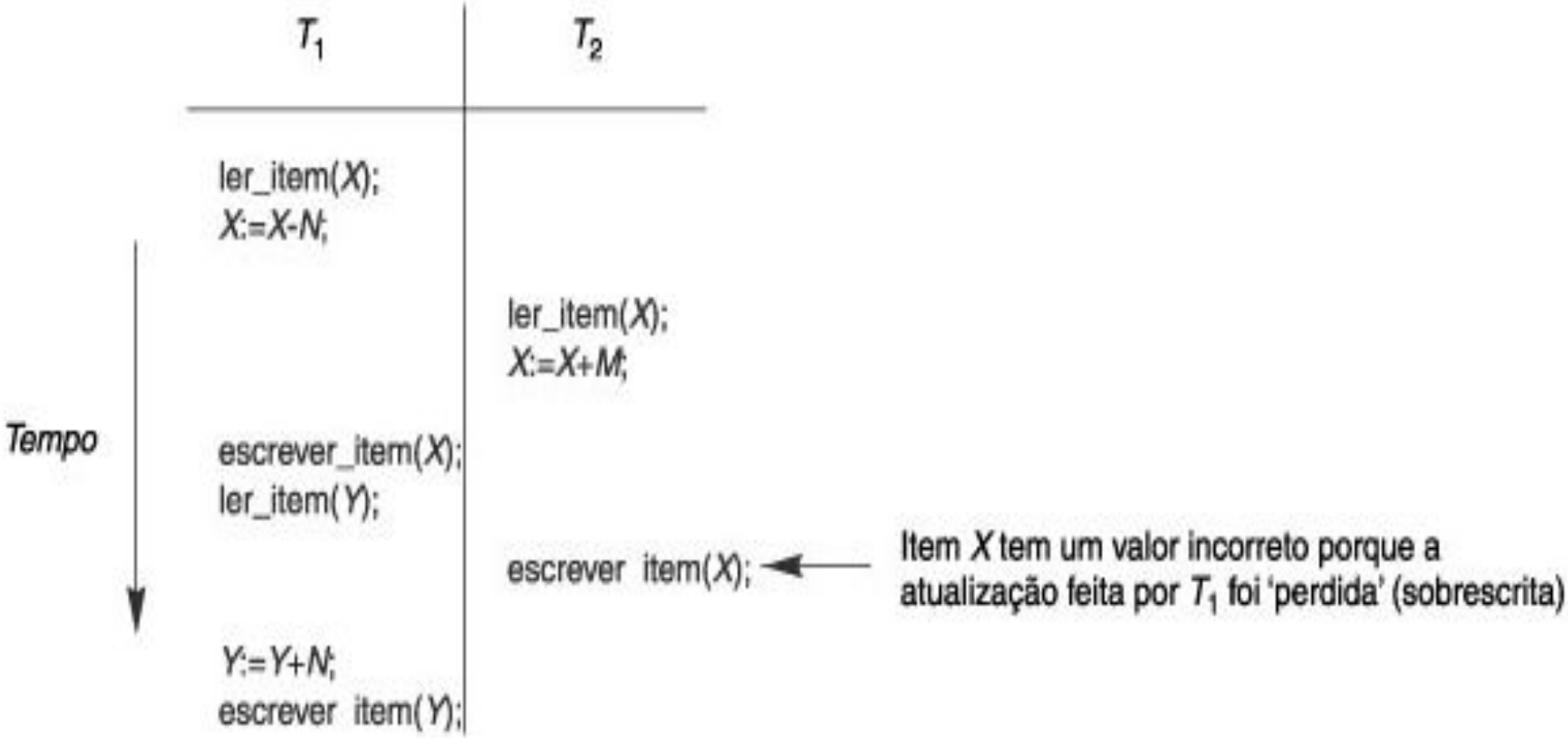
T_1
ler_item (*X*);
X:=*X*-*N*;
escrever_item (*X*);
ler_item (*Y*);
Y:=*Y*+*N*;
escrever_item (*Y*);

(b)

T_2
ler_item (*X*);
X:=*X*+*M*;
escrever_item (*X*);

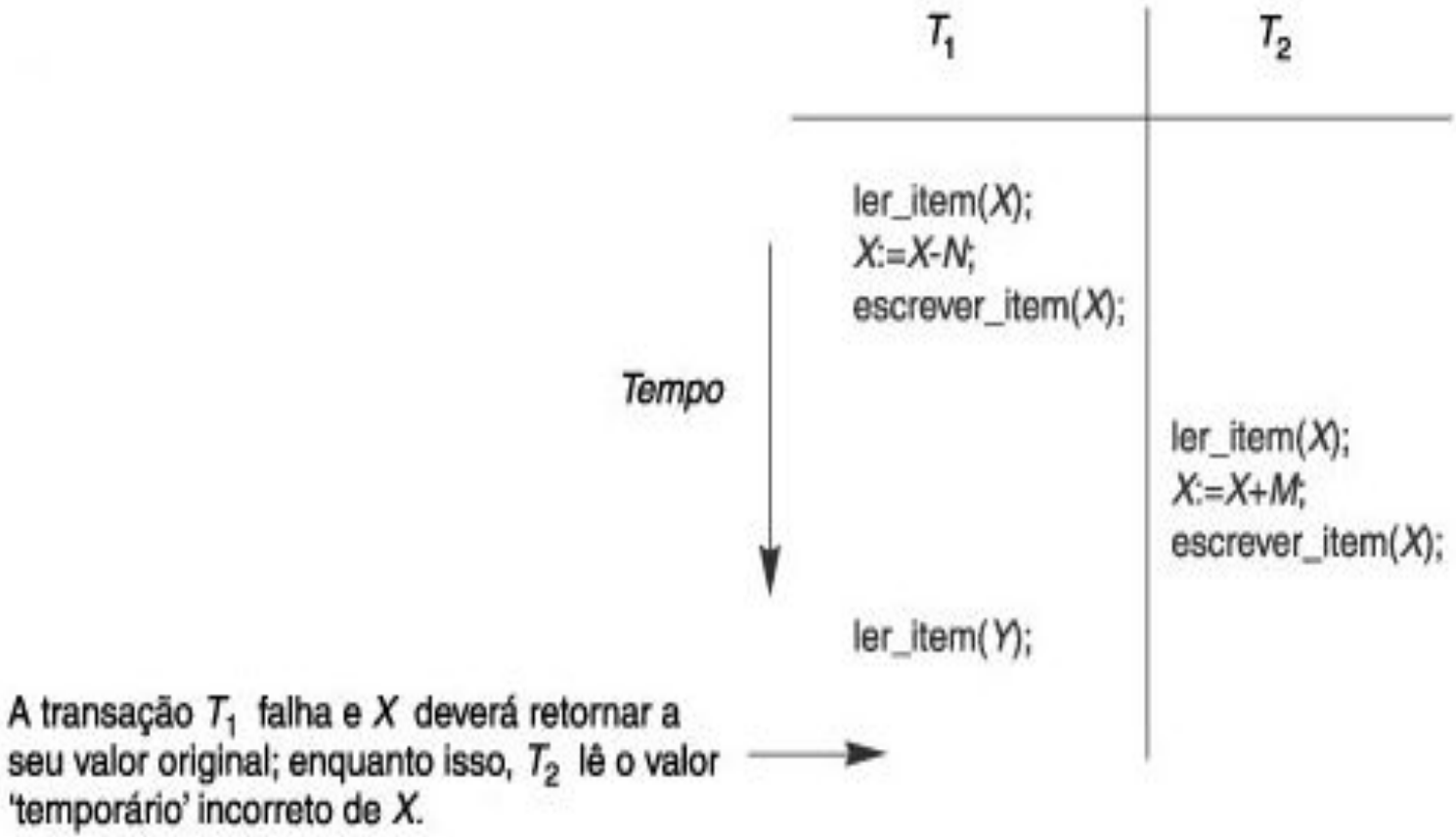
Concorrência

- **Problema da atualização perdida**
  - O valor final de X será incorreto no final da execução.



Concorrência

- **Problema da atualização temporária**
  - Leitura de sujeira



Concorrência

- **Problema do sumário incorreto**

T_1	T_3

	sum:=0;
	ler_item(*A*);
	sum:=*sum*+*A*;
	⋮

ler_item(*X*);
X:=*X*-N;
escrever_item(*X*);	

	ler_item(*X*);
	sum:=*sum*+*X*;
	ler_item(*Y*);
	sum:=*sum*+*Y*;

ler_item(*Y*);
Y:=*Y*+*N*;
escrever_item(*Y*);	

← T_3 lê *X* depois da subtração de *N* e lê *Y* antes da adição de *N*; o resultado é um sumário errado (sem *N*).

Recuperação

- **Possíveis causas de falha**
  - Erro de transação ou de sistema;
  - Erros locais ou condições de exceção detectadas pela transação;
  - Imposição do controle de concorrência;
  - Falha de disco;
  - Problemas físicos em dispositivos de hardware;

Log

- Mecanismo utilizado na recuperação de falhas.

- **Exemplo**
  - [inicio_transação, T]
  - [escrever_item, T, X, valor_antigo, valor_novo]
  - [ler_item, T, X]
  - [commit, T]
  - [abort, T]

Log

- **Exemplo**

  T1: read(A);
  A : = A-100;
  write(A);

- **Sendo A com valor inicial R$ 150,00, o log da transação é**

  < inicio_transacao, T1>
  <escreve_item, T1, A, 150,50>
  <commit, T1>

Planos de Execução

- **Processamento de transações concorrentes**
  - Plano de execução
    - Restaurabilidade
    - Serialidade
      - Plano serial
      - Plano não-serial

Plano de Execução

- **Plano de execução**
  - Para cada operação T_i que participe de um plano S, as operações de T_i em S deverão aparecer na mesma ordem em que elas ocorrem em T_i.

- **Possíveis situações de conflito entre operações**
  - Item de dado X presente em diferentes transações;
  - Acesso ao item X, em diferentes operações;
  - Pelo menos uma das operações é de **escrita**;

Exemplo de Plano de Execução

- S: r1(X); **<u>r2(X)</u>**; w1(X); r1(Y); **<u>w2(X)</u>**; w1(Y)

T_1	T_2
ler_item(*X*);	
X:=*X*-*N*;	
	ler_item(*X*);
	X:=*X*+*M*;
escrever_item(*X*);	
ler_item(*Y*);	
	escrever item(*X*); ← Item *X* tem um valor incorreto porque a atualização feita por T_1 foi 'perdida' (sobrescrita)
Y:=*Y*+*N*;	
escrever item(*Y*);	

Tempo ↓

Planos de Execução

- **Processamento de transações concorrentes**
  - Plano de execução
    - **Restaurabilidade**
    - Serialidade
      - Plano serial
      - Plano não-serial

Plano de Execução - Restaurabilidade

- **Plano de execução completo**
  - As operações em S são exatamente as operações de T_1, T_2,..., T_n, tendo um **commit** ou um **abort** como **última operação** de cada transação do plano.
  - Para quaisquer pares de operações da mesma transação T_i, sua ordem de aparecimento em S será a mesma que em T_i.
  - Para quaisquer duas operações conflitantes, uma das duas precisa aparecer antes da outra no plano.
    - Deve haver uma ordenação para que os resultados sejam corretos.
    - Nas não-conflitantes, isto não é necessário.

Plano de Execução - Restaurabilidade

- Se ocorrer uma falha, **como** um plano pode ser recuperado?

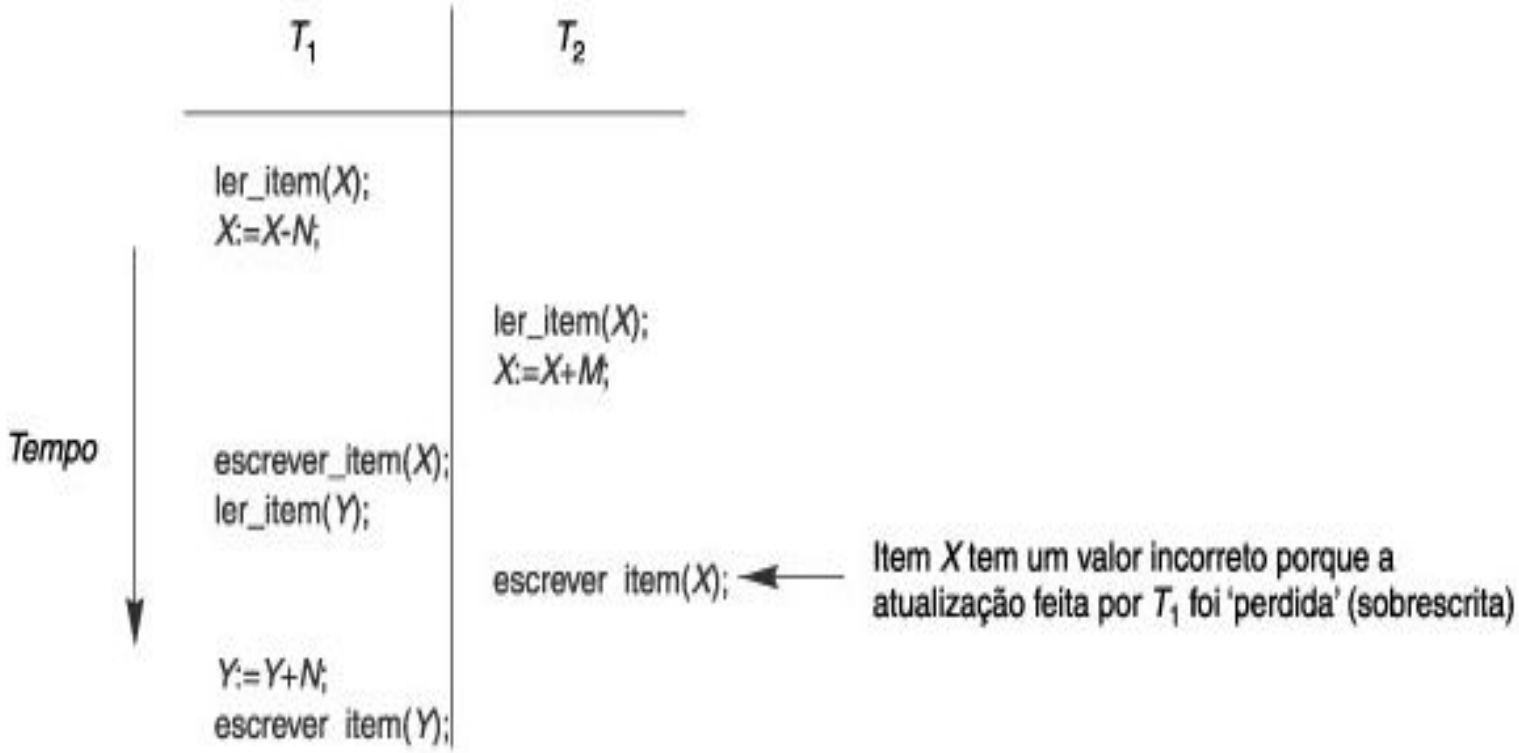
- **Plano restaurável**
  - Deve garantir que uma vez **efetivada** uma transação T, nunca mais seja necessário revertê-la.

Plano de Execução - Restaurabilidade

- **Plano restaurável**
  - Um plano S restaurável é aquele em que, para cada par de transações T_i e T_j em que T_j leia um item de dados previamente escrito por T_i, a operação **commit** de T_i apareça antes da operação de **commit** de T_j.
  - Algoritmo para restauração
    - Usando log;
  - Restaurável
    - S: r1(X); w1(X); **r2(X)**; r1(Y); **w2(X)**; w1(X); c1; **c2**

Plano de Execução - Restaurabilidade

- **Plano não restaurável**
  - S: r1(X); **<u>r2(X)</u>**; w1(X); r1(Y); **<u>w2(X)</u>**; **<u>c2</u>**; w1(Y); **<u>a1</u>**



Plano de Execução - Restaurabilidade

- **Plano restaurável**
  - Reversão **livre** de cascata.
    - Cada transação do plano **lê** somente itens que já foram gravados por transações efetivadas.
    - T_i lê um item gravado por T_j
      - a operação de leitura de T_i vem depois da operação commit de T_j.
  - Reversão **que não é livre** de cascata.

Plano de Execução - Restaurabilidade

- **Plano restrito**
  - Uma transação não pode **nem ler e nem gravar** um item X, até que a última transação do plano que grave X tenha sido **efetivada** ou **abortada**.
  - Plano não-restrito
    - Sa: w1(X,5); **<u>w2(X,8)</u>**; a1
  - Plano restrito
    - Sa: w1(X,5); c1; **w2(X,8)**;

Planos de Execução

- **Processamento de transações concorrentes**
  - Plano de execução
    - Restaurabilidade
    - **Serialidade**
      - Plano serial
      - Plano não-serial

Plano de Execução - Serialidade

- Quando um plano intercalado é um plano serializável?
  - **Plano serial**
    - Não existe intercalação entre as operações das transações.
    - Limitam a concorrência.
  - **Plano não-serial**
    - Apenas uma transação ativa por vez.
    - Serializável: equivalente ao serial.
    - Conflito serialidade.

Exemplo

(a)

T_1	T_2
ler_item(X);	
X:X–N;	
escrever_item(X)	
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	
	ler_item(X);
	X:=X+M;
	escrever_item(X);

Tempo

Plano *A*

(b)

T_1	T_2
	ler_item(X);
	X:=X+M;
	escrever_item(X);
ler_item(X);	
X:=X–N;	
escrever_item(X)	
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	

Tempo

Plano *B*

(c)

T_1	T_2
ler_item(X);	
X:X–N;	
	ler_item(X);
	X:=X+M;
escrever_item(X)	
ler_item(Y)	
	escrever_item(X);
Y:=Y+N;	
escrever_item(Y)	

Tempo

Plano *C*

T_1	T_2
ler_item(X);	
X:=X–N;	
escrever_item(X)	
	ler_item(X);
	X:=X+M;
	escrever_item(X);
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	

Plano *D*

Plano de Execução - Serialidade

- **Conflito Serialidade**
  - Haverá conflito quando houver operação de **escrita**.
  - Duas operações em um plano estarão em **conflito** se elas pertencerem a **transações diferentes**, acessarem o mesmo **item do banco de dados** e pelo menos uma das duas for uma operação **escrever_item**.
  - Um plano S é conflito serializável se ele for equivalente a um plano serial S'.

Plano de Execução – Conflito Serialidade

Algoritmo para testar conflito na serialidade de um plano S

a. Para cada transação $\mathbf{T_i}$ participante do plano S, crie um nó rotulado com $\mathbf{T_i}$ no grafo de precedência.

b. Para cada caso em S onde $\mathbf{T_j}$ executa um read_item(X) depois de $\mathbf{T_i}$ executar um write_item(X), crie uma aresta ($\mathbf{T_i}$ **->** $\mathbf{T_j}$) no grafo de precedência.

c. Para cada caso em S onde $\mathbf{T_j}$ executa um write_item(X) após de $\mathbf{T_i}$ executar um read_item(X), crie uma aresta ($\mathbf{T_i}$ **->** $\mathbf{T_j}$) no grafo de precedência.

d. Para cada caso em S onde $\mathbf{T_j}$ executa um write_item(X) após de $\mathbf{T_i}$ executar um write_item(X), crie uma aresta ($\mathbf{T_i}$ **->** $\mathbf{T_j}$) no grafo de precedência.

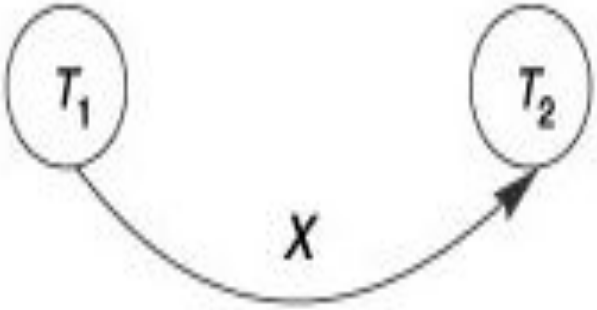
e. O plano S é serializável se, e somente se, o grafo de precedência não possuir ciclos.

Ciclo: a sequência começa e termina no mesmo nó.

Plano de Execução – Conflito Serialidade

T_1	T_2
ler_item(X);	
X:X–N;	
escrever_item(X)	
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	
	ler_item(X);
	X:=X+M;
	escrever_item(X);

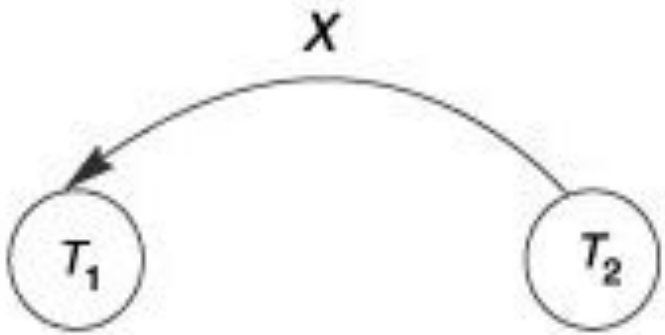
Tempo



Plano de Execução – Conflito Serialidade

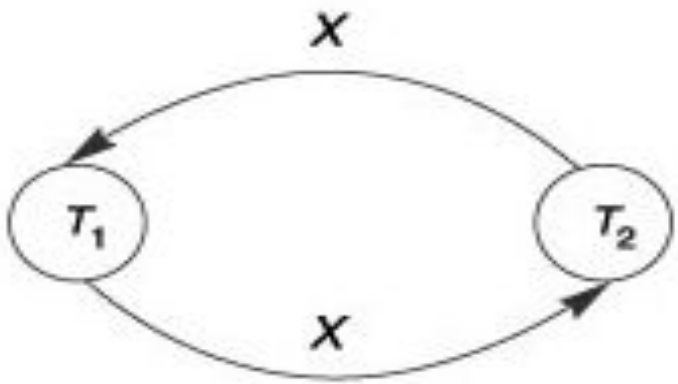
T_1	T_2
	ler_item(X);
	X:=X+M;
	escrever_item(X);
ler_item(X);	
X:=X–N;	
escrever_item(X)	
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	

Tempo



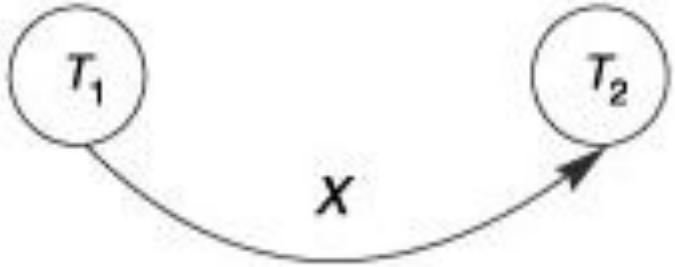
Plano de Execução – Conflito Serialidade

	T_1	T_2
	ler_item(X);	
	X:X–N;	
		ler_item(X);
		X:=X+M;
Tempo	escrever_item(X)	
	ler_item(Y)	
		escrever_item(X);
	Y:=Y+N;	
	escrever_item(Y)	



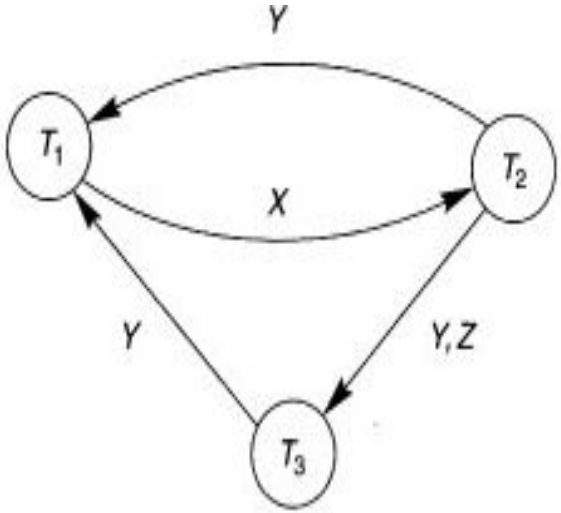
Plano de Execução – Conflito Serialidade

T_1	T_2
ler_item(X);	
X:=X–N;	
escrever_item(X)	
	ler_item(X);
	X:=X+M;
	escrever_item(X);
ler_item(Y)	
Y:=Y+N;	
escrever_item(Y)	



Plano de Execução – Conflito Serialidade

	transação T_1	transação T_2	transação T_3
		ler_item (Z) ler_item (Y) escrever_item (Y);	
			ler_item (Y) ler_item (Z)
Tempo ↓	ler_item (X) escrever_item (X);		escrever_item (Y); escrever_item (Z);
		ler_item (X);	
	ler_item (Y) escrever_item (Y);		
		escrever item (X);	



Planos seriais equivalentes

Nenhuma

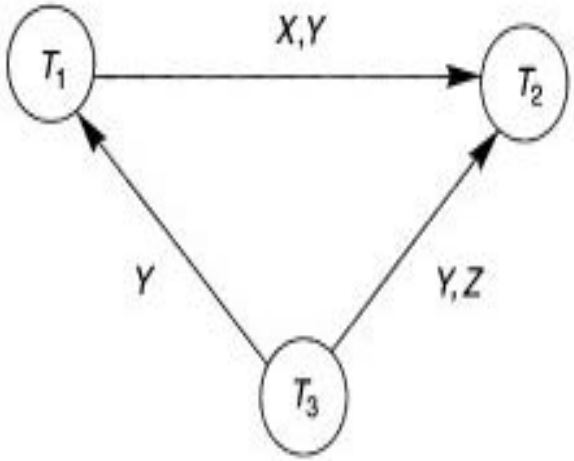
Razão

ciclo $X(T_1 \rightarrow T_2)$, $Y(T_2 \rightarrow T_1)$
ciclo $X(T_1 \rightarrow T_2)$, $YZ(T_2 \rightarrow T_3)$, $Y(T_3 \rightarrow T_1)$

Plano de Execução – Conflito Serialidade

transação T_1	transação T_2	transação T_3
		ler_item (Y) ler_item (Z)
ler_item (X) escrever_item (X);		escrever_item (Y); escrever_item (Z);
	ler_item (Z)	
ler_item (Y) escrever_item (Y);		
	ler_item (Y) escrever_item (Y); ler_item (X); escrever_item (X);	

Tempo



Planos seriais equivalentes

$T_3 \rightarrow T_1 \rightarrow T_2$

Considerações Finais

- Neste material, as **etapas do processamento de transações** foram apresentadas.

- Diferentes **transações** são **executadas** e podem ser comportar de **modo concorrente**.

- O **processamento de transações concorrentes** exige a **preocupação** com a **restaurabilidade** e a **serialidade**. Nesse contexto, **planos de execução** são necessários.

Referência

- Sistemas de Banco de Dados. Elsmari e Navathe. Pearson-Addison Wesley, Quarta Edição.